

NVIDIA Enterprise Stack Demo Guide

Single-Node AWS EKS Cluster with GPU Instance

Author: Nolan **Date:** April 2026 **Version:** 1.0

Document Overview

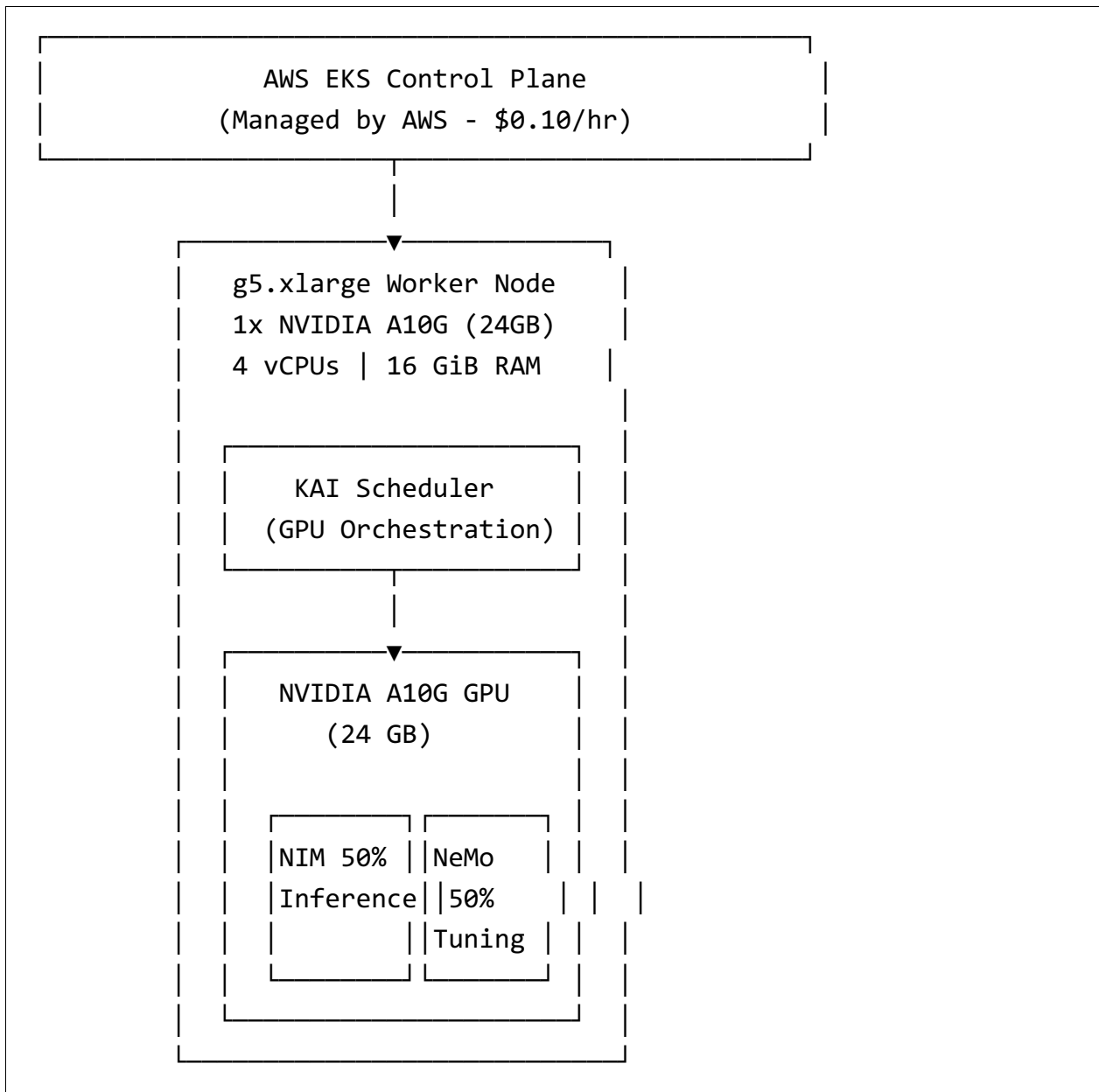
This guide provides a complete, step-by-step script for building and presenting a demo of the NVIDIA Enterprise Stack on a single-node Amazon EKS cluster with a GPU instance. Every NVIDIA component used in this demo is free — your only costs are AWS infrastructure.

Demo Components:

- **KAI Scheduler** — Open-source GPU workload scheduling (Apache 2.0)
- **NVIDIA NIM** — Inference microservices (free for Developer Program members)
- **NVIDIA NeMo Framework** — Model fine-tuning (fully open source)

Target Instance: g5.xlarge (1x NVIDIA A10G GPU, 24GB GPU memory)

Architecture Overview



How it works:

- The EKS control plane is fully managed by AWS
- Your single g5.xlarge instance serves as the sole worker node
- KAI Scheduler manages fractional GPU allocation across pods
- NIM and NeMo share the GPU via KAI Scheduler's fractional GPU annotations

Cost Summary

Component	Type	Cost	Notes
KAI Scheduler	NVIDIA	FREE	Apache 2.0 open source
NVIDIA NIM	NVIDIA	FREE	Free via Developer Program
NVIDIA NeMo	NVIDIA	FREE	Fully open source
NGC Account	NVIDIA	FREE	Free developer account
EKS Control Plane	AWS	\$0.10/hour (\$73/month)	AWS-managed
g5.xlarge Instance	AWS	\$1.006/hour (~\$24/day)	On-demand pricing
EBS Storage (200GB)	AWS	~\$16/month	gp3 volume
1-Day Demo Total		~\$27	Tear down after demo
1-Week Dev Total		~\$190	Running 24/7

Prerequisites Checklist

Before starting, ensure you have the following:

- ☐ **AWS CLI** installed and configured with credentials
 - Verify: `aws --version`
 - Configure: `aws configure`
- ☐ **kubectl** installed (version 1.28+)
 - Verify: `kubectl version --client`
 - Install (macOS): `brew install kubectl`

- ☐ **eksctl** installed (latest version)
 - Verify: `eksctl version`
 - Install (macOS): `brew install eksctl`
 - ☐ **Helm 3.x** installed
 - Verify: `helm version`
 - Install (macOS): `brew install helm`
 - ☐ **git** installed
 - Verify: `git --version`
 - ☐ **NVIDIA Developer Program account** (free)
 - Sign up: <https://developer.nvidia.com/developer-program>
 - ☐ **NGC API Key** generated
 - Generate: <https://org.ngc.nvidia.com/setup/api-keys>
 - When creating, ensure "NGC Catalog" is selected under "Services Included"
 - ☐ **AWS service quota** for g5.xlarge in your target region
 - Check: AWS Console → Service Quotas → EC2 → "Running On-Demand G and VT instances"
 - You need at least 4 vCPUs of quota
-

Step 1: Create the Single-Node GPU EKS Cluster

Estimated time: 15-20 minutes

1.1 Create the Cluster Configuration File

Create a file named `cluster-config.yaml`:

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
```

```
metadata:
```

```
  name: nvidia-demo-cluster
  region: us-west-2
  version: "1.31"
```

```
managedNodeGroups:
```

```
  - name: gpu-node
    instanceType: g5.xlarge
    minSize: 1
    maxSize: 1
    desiredCapacity: 1
    volumeSize: 200
    ami: auto
    ssh:
      allow: false
    labels:
      workload-type: gpu
```

1.2 Create the Cluster

```
eksctl create cluster -f cluster-config.yaml
```

This command will:

- Create the EKS control plane (managed by AWS)
- Create a managed node group with one g5.xlarge instance
- Configure kubectl to connect to the new cluster

1.3 Verify the Cluster

```
# Check nodes are ready
kubectl get nodes
# Expected output: 1 node in "Ready" status

# Check node details
kubectl get nodes -o wide
# Should show the g5.xlarge instance

# Verify node labels
kubectl get nodes --show-labels | grep workload-type
```

Talking Point: "We've created a single-node EKS cluster with one NVIDIA A10G GPU. The control plane is fully managed by AWS — we only manage the workloads."

Step 2: Install NVIDIA GPU Operator

Estimated time: 5-10 minutes

The GPU Operator automatically installs NVIDIA drivers, the device plugin, and the container toolkit on your GPU node.

2.1 Add the NVIDIA Helm Repository

```
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update
```

2.2 Install the GPU Operator

```
helm install gpu-operator nvidia/gpu-operator \
  --namespace gpu-operator \
  --create-namespace \
  --set driver.enabled=true \
  --set toolkit.enabled=true
```

2.3 Wait for Installation to Complete

```
# Watch pods until all are Running/Completed (~5 minutes)
kubectl get pods -n gpu-operator -w
```

Wait until you see all pods in Running or Completed state. Key pods to watch for:

- nvidia-driver-daemonset-*
- nvidia-device-plugin-daemonset-*
- nvidia-container-toolkit-daemonset-*

2.4 Verify GPU Detection

```
kubectl get nodes -o=custom-  
columns='NAME:.metadata.name,GPU:.status.allocatable.nvidia\.com/gpu  
'
```

Expected output should show 1 under the GPU column.

Talking Point: "The NVIDIA GPU Operator automatically handles all driver installation and GPU exposure to Kubernetes. No manual driver setup needed."

Step 3: Install KAI Scheduler

Estimated time: 5 minutes

KAI Scheduler is NVIDIA's open-source, Kubernetes-native GPU scheduling solution, released under the Apache 2.0 license. It was originally the core scheduling engine of the Run:ai platform.

3.1 Clone the Repository

```
git clone https://github.com/NVIDIA/KAI-Scheduler.git  
cd KAI-Scheduler
```

3.2 Install via Helm

```
helm install kai-scheduler ./deployments/kai-scheduler \  
  --namespace kai-scheduler \  
  --create-namespace
```

3.3 Verify Installation

```
kubectl get pods -n kai-scheduler
```

All pods should be in Running state.

3.4 Create a Queue for Demo Workloads

```
cat <<EOF | kubectl apply -f -
apiVersion: scheduling.run.ai/v1
kind: Queue
metadata:
  name: demo-queue
spec:
  resources:
    gpu:
      quota: 1
      limit: 1
EOF
```

3.5 Verify the Queue

```
kubectl get queues
```

Talking Point: "KAI Scheduler is the open-source core of NVIDIA's Run:ai platform. It enables fractional GPU allocation — meaning multiple workloads can share a single GPU with memory isolation. This is critical for maximizing GPU utilization in production environments."

Step 4: Set Up NGC Credentials

Estimated time: 2 minutes

4.1 Export Your NGC API Key

```
export NGC_API_KEY="<YOUR_NGC_API_KEY>"
```

Replace <YOUR_NGC_API_KEY> with the key you generated from <https://org.ngc.nvidia.com/setup/api-keys>

4.2 Create Kubernetes Secrets


```
# Secret for pulling NVIDIA container images
kubectl create secret docker-registry ngc-secret \
  --docker-server=nvcr.io \
  --docker-username='${oauth_token}' \
  --docker-password=${NGC_API_KEY}

# Secret for NIM runtime authentication
kubectl create secret generic ngc-api-key \
  --from-literal=NGC_API_KEY=${NGC_API_KEY}
```

4.3 Verify Secrets

```
kubectl get secrets | grep ngc
```

You should see both `ngc-secret` and `ngc-api-key`.

Step 5: Deploy NVIDIA NIM for Inference

Estimated time: 10-15 minutes (model download)

NVIDIA NIM provides optimized inference microservices. As a Developer Program member, you get free access to downloadable NIM containers for development and testing.

For the A10G GPU (24GB), we'll use **Llama 3.1 8B Instruct** which fits comfortably in memory.

5.1 Create the NIM Deployment

Create a file named `nim-deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nim-llm
  labels:
    app: nim-llm
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nim-llm
  template:
    metadata:
      labels:
        app: nim-llm
    annotations:
      gpu-fraction: "0.5"
    spec:
      schedulerName: kai-scheduler
      imagePullSecrets:
        - name: ngc-secret
      containers:
        - name: nim
          image: nvcr.io/nim/meta/llama-3.1-8b-instruct:latest
          resources:
            limits:
              nvidia.com/gpu: 1
          env:
            - name: NGC_API_KEY
              valueFrom:
                secretKeyRef:
                  name: ngc-api-key
                  key: NGC_API_KEY
          ports:
            - containerPort: 8000
              name: http
          readinessProbe:
            httpGet:
              path: /v1/health/ready
```

```
        port: 8000
        initialDelaySeconds: 120
        periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: nim-llm-service
spec:
  selector:
    app: nim-llm
  ports:
    - protocol: TCP
      port: 8000
      targetPort: 8000
  type: ClusterIP
```

5.2 Deploy

```
kubectl apply -f nim-deployment.yaml
```

5.3 Monitor Deployment

```
# Watch pod status – model download takes ~10-15 minutes
kubectl get pods -w

# Check logs for download progress
kubectl logs -f deployment/nim-llm
```

Wait until the pod status shows **Running** and the readiness probe passes.

5.4 Test Inference

```
# Port-forward to access locally
kubectl port-forward svc/nim-llm-service 8000:8000 &

# Test with a chat completion request
curl -s -X POST http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta/llama-3.1-8b-instruct",
    "messages": [
      {"role": "system", "content": "You are a helpful AI
assistant."},
      {"role": "user", "content": "Explain GPU scheduling in
Kubernetes in 3 sentences."}
    ],
    "max_tokens": 200,
    "temperature": 0.7
  }' | python -m json.tool
```

5.5 Additional Test Queries for Demo

```
# Code generation test
curl -s -X POST http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta/llama-3.1-8b-instruct",
    "messages": [
      {"role": "user", "content": "Write a Python function to
calculate Fibonacci numbers"}
    ],
    "max_tokens": 300
  }' | python -m json.tool

# Streaming test
curl -N -X POST http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "meta/llama-3.1-8b-instruct",
    "messages": [
      {"role": "user", "content": "What are the benefits of
fractional GPU utilization?"}
    ],
    "max_tokens": 200,
    "stream": true
  }'
```

Talking Point: "NVIDIA NIM provides production-ready inference microservices that deploy in minutes. The model is automatically optimized for the underlying GPU hardware. We're running this with only 50% of the GPU allocated via KAI Scheduler, leaving room for other workloads."

Step 6: Fine-Tune with NVIDIA NeMo

Estimated time: 15-30 minutes (depending on dataset size)

NeMo Framework is NVIDIA's fully open-source framework for training and fine-tuning generative AI models. It can run on a single GPU with as low as 16GB of GPU RAM. We'll use LoRA (Low-Rank Adaptation) for parameter-efficient fine-tuning.

6.1 Create the Fine-Tuning Job

Create a file named `nemo-finetune-job.yaml`:

```

apiVersion: batch/v1
kind: Job
metadata:
  name: nemo-lora-finetune
spec:
  template:
    metadata:
      annotations:
        gpu-fraction: "0.5"
    spec:
      schedulerName: kai-scheduler
      restartPolicy: Never
      imagePullSecrets:
        - name: ngc-secret
      containers:
        - name: nemo
          image: nvcr.io/nvidia/nemo:24.12.01
          command: ["/bin/bash", "-c"]
          args:
            - |
              echo "=== Step 1: Preparing Dataset ==="
              pip install datasets -q

              python -c "
              from datasets import load_dataset
              import json

              # Download a small subset for demo purposes
              ds = load_dataset('databricks/databricks-dolly-15k',
split='train[:500]')

              # Format for NeMo SFT
              formatted = []
              for item in ds:
                  formatted.append({
                      'input': item.get('context', '') + ' ' +
item['instruction'],
                      'output': item['response']
                  })

```

```

        with open('/workspace/data/train.jsonl', 'w') as f:
            for item in formatted:
                f.write(json.dumps(item) + '
')

        # Create a small validation set
        with open('/workspace/data/val.jsonl', 'w') as f:
            for item in formatted[:50]:
                f.write(json.dumps(item) + '
')

        print(f'Training samples: {len(formatted)}')
        print('Dataset ready!')
        "

        echo "=== Step 2: Starting LoRA Fine-Tuning ==="
        python
/opt/NeMo/examples/nlp/language_modeling/tuning/megatron_gpt_finetuning.py \
            trainer.devices=1 \
            trainer.max_epochs=2 \
            trainer.precision=bf16 \
            model.micro_batch_size=1 \
            model.global_batch_size=4 \
            model.peft.peft_scheme=lora \
            model.peft.lora_tuning.adapter_dim=16 \

model.data.train_ds.file_names=[/workspace/data/train.jsonl] \

model.data.validation_ds.file_names=[/workspace/data/val.jsonl] \
            exp_manager.exp_dir=/workspace/results

        echo "=== Fine-Tuning Complete ==="
        ls -la /workspace/results/
resources:
    limits:
        nvidia.com/gpu: 1
volumeMounts:

```



```
- name: workspace
  mountPath: /workspace
volumes:
- name: workspace
  emptyDir:
    sizeLimit: 50Gi
```

6.2 Submit the Fine-Tuning Job

```
kubectl apply -f nemo-finetune-job.yaml
```

6.3 Monitor Progress

```
# Watch the job status
kubectl get jobs -w

# Follow the logs in real-time
kubectl logs -f job/nemo-lora-finetune

# Check GPU utilization during training
kubectl exec -it $(kubectl get pods -l job-name=nemo-lora-finetune \
  -o jsonpath='{.items[0].metadata.name}') -- nvidia-smi
```

Talking Point: "NeMo Framework is NVIDIA's fully open-source training platform. We're using LoRA — a parameter-efficient fine-tuning method that only trains a small number of adapter parameters instead of the full model. This makes fine-tuning feasible on a single GPU with limited memory. Notice that this job is running alongside our inference workload, both sharing the same GPU via KAI Scheduler."

Step 7: Demonstrate Fractional GPU Sharing

Estimated time: 5 minutes

This is the key differentiator — showing how KAI Scheduler enables multiple AI workloads to share a single GPU.

7.1 Show Current GPU Allocation

```
# View node resource allocation
kubectl describe node $(kubectl get nodes -o
jsonpath='{.items[0].metadata.name}') \
| grep -A 10 "Allocated resources"
```

7.2 Deploy a Second GPU Workload

While NIM is running (using 50% GPU), deploy another workload:

```

cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: gpu-monitor
  annotations:
    gpu-fraction: "0.5"
spec:
  schedulerName: kai-scheduler
  containers:
  - name: pytorch
    image: nvcr.io/nvidia/pytorch:24.01-py3
    command: ["python", "-c"]
    args:
      - |
        import torch
        import time

        print(f"GPU Device: {torch.cuda.get_device_name(0)}")
        print(f"Total GPU Memory:
{torch.cuda.get_device_properties(0).total_mem / 1e9:.1f} GB")
        print(f"Allocated Memory: {torch.cuda.memory_allocated(0) /
1e9:.3f} GB")
        print(f"Reserved Memory: {torch.cuda.memory_reserved(0) /
1e9:.3f} GB")

        # Run a simple computation to show GPU is active
        x = torch.randn(1000, 1000, device='cuda')
        for i in range(10):
            y = torch.matmul(x, x)
            print(f"Iteration {i+1}: Matrix multiplication complete,
"
                f"Allocated: {torch.cuda.memory_allocated(0) /
1e6:.1f} MB")
            time.sleep(2)

        print("Workload complete! GPU was shared successfully.")
        time.sleep(300) # Keep pod alive for inspection
resources:

```

```
    limits:
      nvidia.com/gpu: 1
  imagePullSecrets:
  - name: ngc-secret
EOF
```

7.3 Verify Both Workloads Are Running

```
# Show all GPU-using pods
kubectl get pods -o wide

# Check GPU utilization from the monitor pod
kubectl logs gpu-monitor

# Check nvidia-smi from either pod
kubectl exec -it gpu-monitor -- nvidia-smi
```

7.4 Show the Scheduling in Action

```
# List all pods with their scheduler
kubectl get pods -o=custom-
columns='NAME:.metadata.name,SCHEDULER:.spec.schedulerName,STATUS:.s
tatus.phase'

# Show KAI Scheduler events
kubectl get events --field-selector reason=Scheduled --sort-
by='.lastTimestamp'
```

Talking Point: "Here's the power of KAI Scheduler — we have two workloads running simultaneously on a single GPU. The inference service gets 50% of GPU resources, and our monitoring workload gets the other 50%. In a production environment, this means you can run inference, training, and data processing jobs on the same GPU infrastructure without wasting resources. Without fractional GPU scheduling, each workload would need its own dedicated GPU."

Demo Presentation Script

Recommended Presentation Flow (30-45 minutes)

Opening (5 minutes)

"Today I'm going to demonstrate the NVIDIA Enterprise AI Stack running on AWS. What makes this demo unique is that every NVIDIA component is free and open source — we're only paying for AWS infrastructure. The three key components are:

1. KAI Scheduler — NVIDIA's open-source GPU scheduler for Kubernetes
2. NVIDIA NIM — Optimized inference microservices
3. NVIDIA NeMo — Open-source framework for model fine-tuning

All running on a single g5.xlarge instance with one NVIDIA A10G GPU."

Infrastructure Tour (5 minutes)

Show the EKS cluster and GPU node:

```
kubectl get nodes -o wide
kubectl describe node $(kubectl get nodes -o
jsonpath='{.items[0].metadata.name}') | head -30
kubectl get pods -n gpu-operator
kubectl get pods -n kai-scheduler
```

Inference Demo (10 minutes)

1. Show the NIM deployment and its fractional GPU annotation
2. Run several inference queries demonstrating different capabilities
3. Show GPU memory utilization with nvidia-smi
4. Highlight the OpenAI-compatible API format

Fine-Tuning Demo (10 minutes)

1. Submit the NeMo fine-tuning job
2. Show it running alongside inference (both sharing the GPU)
3. Monitor training progress in the logs
4. Explain LoRA and parameter-efficient fine-tuning

GPU Sharing Demo (10 minutes)

1. Deploy the monitoring workload

2. Show multiple pods sharing the single GPU
3. Display nvidia-smi showing memory partitioning
4. Explain how KAI Scheduler manages fractional allocation

Closing (5 minutes)

"What we've demonstrated today is a complete AI lifecycle on a single GPU:

- Inference serving with NVIDIA NIM
- Model fine-tuning with NVIDIA NeMo
- Intelligent GPU resource management with KAI Scheduler

All using free, open-source NVIDIA tools on AWS infrastructure. This same architecture scales from a single GPU demo to multi-node production clusters."

Troubleshooting

GPU Not Detected

```
# Check GPU Operator pods
kubectl get pods -n gpu-operator

# Check driver installation logs
kubectl logs -n gpu-operator $(kubectl get pods -n gpu-operator \
  -l app=nvidia-driver-daemonset -o
  jsonpath='{.items[0].metadata.name}')
```

```
# Verify node has GPU resource
kubectl describe node $(kubectl get nodes -o
  jsonpath='{.items[0].metadata.name}') \
  | grep nvidia
```

NIM Pod Stuck in Pending

```
# Check pod events
kubectl describe pod $(kubectl get pods -l app=nim-llm -o
jsonpath='{.items[0].metadata.name}')
```

Common causes:

- # - NGC credentials incorrect → recreate ngc-secret
- # - Insufficient GPU memory → check no other GPU workloads running
- # - Image pull error → verify NGC_API_KEY is valid

NIM Pod CrashLoopBackOff

```
# Check logs for error details
kubectl logs $(kubectl get pods -l app=nim-llm -o
jsonpath='{.items[0].metadata.name}') --previous
```

Common causes:

- # - Model too large for GPU memory → use a smaller model
- # - NGC API key expired → regenerate and update secret

KAI Scheduler Not Scheduling Pods

```
# Check KAI Scheduler logs
kubectl logs -n kai-scheduler $(kubectl get pods -n kai-scheduler \
-o jsonpath='{.items[0].metadata.name}')
```

Verify queue exists

```
kubectl get queues
```

Check if pods specify the correct schedulerName

```
kubectl get pod <pod-name> -o jsonpath='{.spec.schedulerName}'
```

NeMo Fine-Tuning Out of Memory

```
# Reduce batch size in the job YAML:  
# model.micro_batch_size=1  
# model.global_batch_size=2  
  
# Or reduce LoRA adapter dimension:  
# model.peft.lora_tuning.adapter_dim=8
```

General Debugging Commands

```
# Check all pod statuses  
kubectl get pods --all-namespaces  
  
# Check cluster events  
kubectl get events --sort-by='.lastTimestamp' | tail -20  
  
# Check node resource usage  
kubectl top nodes  
kubectl top pods
```

Cleanup

Important: Run these commands after your demo to stop all AWS charges.


```

# Step 1: Delete all workloads
kubectl delete -f nim-deployment.yaml 2>/dev/null
kubectl delete -f nemo-finetune-job.yaml 2>/dev/null
kubectl delete pod gpu-monitor 2>/dev/null

# Step 2: Uninstall Helm releases
helm uninstall kai-scheduler -n kai-scheduler 2>/dev/null
helm uninstall gpu-operator -n gpu-operator 2>/dev/null

# Step 3: Delete the entire EKS cluster
# This removes the control plane, node group, and associated
resources
eksctl delete cluster --name nvidia-demo-cluster --region us-west-2

# Step 4: Verify cleanup
aws eks list-clusters --region us-west-2
# Should not show nvidia-demo-cluster

```

Note: Cluster deletion takes approximately 10-15 minutes. Verify in the AWS Console that all associated resources (EC2 instances, EBS volumes, VPC) have been cleaned up.

Appendix: Useful Links

Resource	URL
KAI Scheduler GitHub	https://github.com/NVIDIA/KAI-Scheduler
NVIDIA NIM Documentation	https://docs.nvidia.com/nim/
NVIDIA NeMo Framework	https://github.com/NVIDIA-NeMo/NeMo
NeMo User Guide	https://docs.nvidia.com/nemo-framework/user-guide/latest/

Resource	URL
NVIDIA Developer Program	https://developer.nvidia.com/developer-program
NGC API Key Setup	https://org.ngc.nvidia.com/setup/api-keys
AWS EKS Documentation	https://docs.aws.amazon.com/eks/
eksctl Documentation	https://eksctl.io/
NVIDIA GPU Operator	https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/
AWS GPU Instance Types	https://aws.amazon.com/ec2/instance-types/#Accelerated_Computing
AWS Blog: NIM on EKS	https://aws.amazon.com/blogs/hpc/deploying-generative-ai-applications-with-nvidia-nims-on-amazon-eks/
AWS Blog: Run:ai on EKS	https://aws.amazon.com/blogs/containers/maximizing-gpu-utilization-using-nvidia-runai-in-amazon-eks/